

Dynamic Programming

Fibonacci numbers

$F(n)$

$F(n-1) + F(n-2)$

if $n=0$
if $n=1$
if $n>1$

0, 1, 1, 2, 3, 5, ...
0 1 2 3 4 5

$F(n) :$

if $n=0 :$

return 0

if $n=1 :$

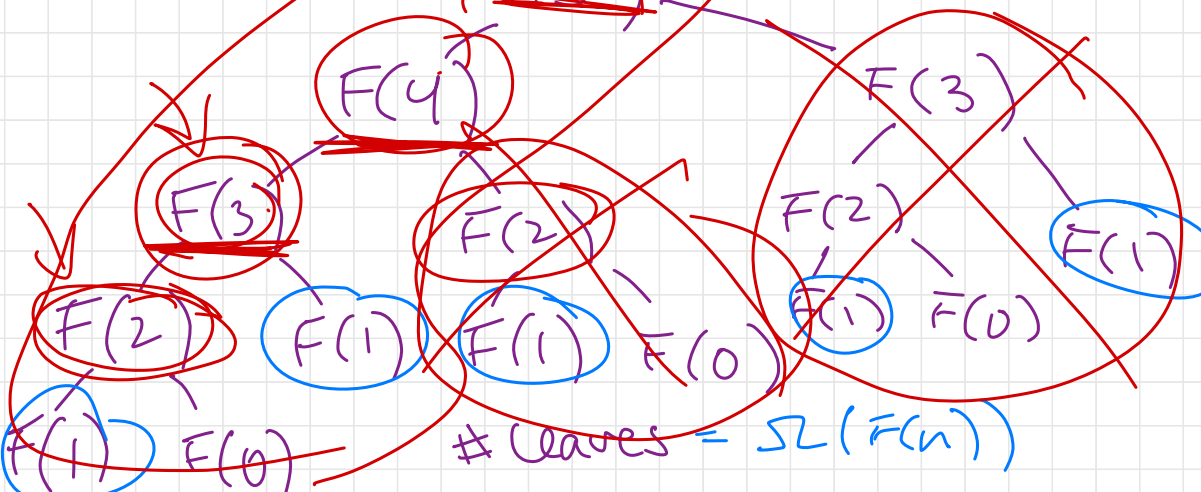
return 1

else:

return $F(n-1) + F(n-2)$

look up in table
if already
filled

$F(5) = 5$

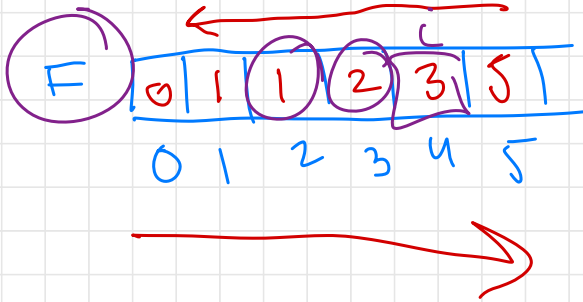


leaves = $2(F(n))$

Memoization

nodes = $\mathcal{O}(2^n)$
↑
upper bound

memoize in an array



$F(5)$

Fibonacci(n):

initialize array $F[0..n]$

$F[0] = 0$

$F[1] = 1$

for i in 2 to n :

$F[i] = F[i-1] + F[i-2]$

return $F[n]$

Text Segmentation problem

input $A[1..n]$ characters BLUESTEM
is A splittable into words?

"word(i, j) = T iff $A[i, j]$ is a word"

recursive subproblems

$\text{Splittable}(i) = \text{True}$ iff $A[i..n]$ can be split into words



$A[i]$

```
Splittable(i):  
  if  $i > n$ :  
    return T
```

else:

```
  for  $j$  in  $i+1$  to  $n$ :
```

```
    if isword( $i, j$ ):  
      return Splittable( $j$ )
```

```
  else:  
    return F
```

memoize splittable alg into an array.

$\text{SplitTable}[0..n+1]$

$\text{SplitTable}[i] = \text{splittable}(i)$



to fill in element i

① What is the data type?

② What other array indices do I use to compute $\text{SplitTable}[i]$?

$\text{SplitTable}[i]$ depends on $\text{SplitTable}[j], j > i$

$A[1..n]$

$\text{SplitTable}(1) :$

$\text{SplitTable}[n+1] = \text{True}$

for i in n to 1 :

$\text{SplitTable}[i] = F$

for j in i to n :

if $\text{IsWord}(i, j)$ and $\text{SplitTable}[j]$:

$\text{SplitTable}[i] = T$

return $\text{SplitTable}[1]$

$O(n^2)$